

# Assignment #B: 图为主

Updated 2223 GMT+8 Apr 29, 2025

2025 spring, Compiled by 李皓翔 物院

## 说明:

### 1. 解题与记录:

对于每一个题目, 请提供其解题思路(可选), 并附上使用Python或C++编写的源代码(确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted)。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。(推荐使用Typora <https://typoraio.cn> 进行编辑, 当然你也可以选择Word。)无论题目是否已通过, 请标明每个题目大致花费的时间。

2. **提交安排:** 提交时, 请首先上传PDF格式的文件, 并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的头像, 提交的文件为PDF格式, 并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交:** 如果你预计无法在截止日期前提交作业, 请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业, 以保证顺利完成课程要求。

## 1. 题目

### E07218:献给阿尔吉侬的花束

bfs, <http://cs101.openjudge.cn/practice/07218/>

思路:

代码:

```
from collections import deque

def bfs(g,q,m):
    vd=[[True for i in range(C+2)] for j in range(R+2)]
    vd[sx][sy]=False
    ans=0
    f=False
    while q:
        for _ in range(len(q)):
            x,y=q.popleft()
            if (x,y)==g:
                return ans
            for i,j in [(1,0),(-1,0),(0,1),(0,-1)]:
                nx=x+i
                ny=y+j
                if vd[nx][ny] and m[nx][ny]:
                    q.append((nx,ny))
                    vd[nx][ny]=False
```

```

        ans+=1
    return -1

for _ in range(int(input())):
    R,C=map(int,input().split())
    m=[[False for i in range(C+2)] for j in range(R+2)]
    for i in range(R):
        tem=input()
        for j in range(C):
            if tem[j]=='.':
                m[i+1][j+1]=True
            if tem[j]=='S':
                q=deque([(i+1,j+1)])
                sx=i+1
                sy=j+1
            if tem[j]=='E':
                m[i+1][j+1]=True
                g=(i+1,j+1)
    r=bfs(g,q,m)
    if r!=-1:
        print(r)
    else:
        print('oop!')

```

代码运行截图 (至少包含有"Accepted")

#49071158提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```

from collections import deque

def bfs(g,q,m):
    vd=[[True for i in range(C+2)] for j in range(R+2)]
    vd[sx][sy]=False
    ans=0
    f=False
    while q:
        for _ in range(len(q)):
            x,y=q.popleft()
            if (x,y)==g:
                return ans
            for i,j in [(1,0),(-1,0),(0,1),(0,-1)]:
                nx=x+i
                ny=y+j
                if vd[nx][ny] and m[nx][ny]:
                    q.append((nx,ny))
                    vd[nx][ny]=False
        ans+=1
    return -1

for _ in range(int(input())):
    R,C=map(int,input().split())
    m=[[False for i in range(C+2)] for j in range(R+2)]
    for i in range(R):

```

基本信息

#: 49071158  
 题目: 07218  
 提交人: 24n2400011450  
 内存: 4036kB  
 时间: 85ms  
 语言: Python3  
 提交时间: 2025-05-06 09:47:16

## M3532.针对图的路径存在性查询I

disjoint set, <https://leetcode.cn/problems/path-existence-queries-in-a-graph-i/>

思路:

代码:

```
class Solution:
    def pathExistenceQueries(self, n: int, nums: List[int], maxDiff: int,
queries: List[List[int]]) -> List[bool]:
        parent=list(range(n))
        def find(x):
            if parent[x]!=x:
                parent[x]=find(parent[x])
            return parent[x]
        def union(x, y):
            fx, fy = find(x), find(y)
            if fx != fy:
                fx, fy = max(fx, fy), min(fx, fy)
                parent[fx] = fy
        m=0
        for i in range(n):
            b=bisect_right(nums,nums[i]+maxDiff)-1
            for j in range(max(m,i),b+1):
                union(i,j)
            m=max(m,b)

        for i in range(n):
            find(i)
        ans=[]
        for x,y in queries:
            tmp=(find(x)==find(y))
            ans.append(tmp)
        return ans
```

代码运行截图 (至少包含有"Accepted")

通过 550 / 550 个通过的测试用例

Moon Velvet 提交于 2025.05.06 21:26

写题解

从无序中  
寻找有序

第 447 场力扣周赛

「微观博易」专场，参与竞赛可获企业内推机会及精美周边，期待你的加入！



🕒 执行用时分布



1280 ms | 击败 5.13%

🌟 复杂度分析

💾 消耗内存分布

51.46 MB | 击败 30.62%



## M22528:厚道的调分方法

binary search, <http://cs101.openjudge.cn/practice/22528/>

思路:

代码:

```
def find_min_b(scores):
    n = len(scores)
    required = 0.6 * n
    left, right = 1, 10**9
    answer = 10**9

    while left <= right:
        mid = (left + right) // 2
        a = mid / 10**9
        count = 0
        for x in scores:
            adjusted = a * x + (1.1 ** (a * x))
            if adjusted >= 85:
                count += 1
        if count >= required:
            answer = mid
            right = mid - 1
        else:
```

```
        left = mid + 1
    return answer
```

```
scores = [float(x) for x in input().split()]
print(find_min_b(scores))
```

代码运行截图 (至少包含有"Accepted")

## #49077943提交状态

状态: Accepted

源代码

```
def find_min_b(scores):
    n = len(scores)
    required = 0.6 * n
    left, right = 1, 10**9
    answer = 10**9

    while left <= right:
        mid = (left + right) // 2
        a = mid / 10**9
        count = 0
        for x in scores:
            adjusted = a * x + (1.1 ** (a * x))
            if adjusted >= 85:
                count += 1
        if count >= required:
            answer = mid
            right = mid - 1
        else:
            left = mid + 1
    return answer

scores = [float(x) for x in input().split()]
print(find_min_b(scores))
```

## Msy382: 有向图判环

dfs, <https://sunnywhy.com/sfbj/10/3/382>

思路:

代码:

```
def is_cyclic(u, graph, vis):
    vis[u] = 0
```

```

    for v in graph[u]:
        if vis[v] == -1:
            if is_cyclic(v, graph, vis):
                return True
        elif vis[v] == 0:
            return True
    vis[u] = 1
    return False

import sys
input = sys.stdin.read
data = input().split()

n = int(data[0])
m = int(data[1])

graph = [[] for _ in range(n)]
vis = [-1] * n

index = 2
for _ in range(m):
    u = int(data[index])
    v = int(data[index + 1])
    graph[u].append(v)
    index += 2

f=True
for i in range(n):
    if vis[i] == -1:
        if is_cyclic(i, graph, vis):
            print("Yes")
            f=False
            break

if f:
    print("No")

```

代码运行截图 (至少包含有"Accepted")

代码书写

```
1 def is_cyclic(u, graph, vis):
2     vis[u] = 0
3     for v in graph[u]:
4         if vis[v] == -1:
5             if is_cyclic(v, graph, vis):
6                 return True
7         elif vis[v] == 0:
8             return True
9     vis[u] = 1
10    return False
11
12 import sys
13 input = sys.stdin.read
14 data = input().split()
15
16 n = int(data[0])
17 m = int(data[1])
18
19 graph = [[] for _ in range(n)]
20 vis = [-1] * n
21
22 index = 2
23 for _ in range(m):
24     u = int(data[index])
25     v = int(data[index + 1])
26     graph[u].append(v)
```

测试输入

提交结果

历史提交

完美通过

100% 数据通过测试 [详情](#)

运行时长: 0 ms

## M05443:兔子与樱花

Dijkstra, <http://cs101.openjudge.cn/practice/05443/>

思路:

代码:

```

import heapq

def dijkstra(graph, start, end):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    predecessors = {node: None for node in graph}

    heap = [(0, start)]

    while heap:
        current_distance, current_node = heapq.heappop(heap)

        if current_distance > distances[current_node]:
            continue

        if current_node == end:
            break

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                predecessors[neighbor] = current_node
                heapq.heappush(heap, (distance, neighbor))

    path = []
    current = end
    while current is not None:
        path.append(current)
        current = predecessors[current]
    path.reverse()

    return path, distances

def main():
    P = int(input())
    locations = [input().strip() for _ in range(P)]

    Q = int(input())
    graph = {loc: {} for loc in locations}
    for _ in range(Q):
        src, dest, dist = input().split()
        dist = int(dist)
        graph[src][dest] = dist
        graph[dest][src] = dist

    R = int(input())
    queries = [input().split() for _ in range(R)]

    for query in queries:
        start, end = query
        if start == end:
            print(start)
            continue

        path, distances = dijkstra(graph, start, end)

```

```

output = []
for i in range(len(path)-1):
    current = path[i]
    next_node = path[i+1]
    distance = graph[current][next_node]
    output.append(f"{current}->({distance})")
output.append(path[-1])

print("->".join(output))

if __name__ == "__main__":
    main()

```

代码运行截图 (至少包含有"Accepted")

## #49077984提交状态

状态: Accepted

去

源代码

```

import heapq

def dijkstra(graph, start, end):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    predecessors = {node: None for node in graph}

    heap = [(0, start)]

    while heap:
        current_distance, current_node = heapq.heappop(heap)

        if current_distance > distances[current_node]:
            continue

        if current_node == end:
            break

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                predecessors[neighbor] = current_node
                heapq.heappush(heap, (distance, neighbor))

```

## T28050: 骑士周游

dfs, <http://cs101.openjudge.cn/practice/28050/>

思路:

代码:

```

def knight_tour():
    n = int(input())
    sr, sc = map(int, input().split())
    moves = [(-2, 1), (-1, 2), (1, 2), (2, 1),
              (2, -1), (1, -2), (-1, -2), (-2, -1)]
    visited = [[False] * n for _ in range(n)]
    total_steps = n * n

    def get_degree(x, y):
        count = 0
        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < n and 0 <= ny < n and not visited[nx][ny]:
                count += 1
        return count

    def backtrack(x, y, step):
        if step == total_steps:
            return True
        candidates = []
        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < n and 0 <= ny < n and not visited[nx][ny]:
                degree = get_degree(nx, ny)
                candidates.append((degree, nx, ny))
        candidates.sort()
        for _, nx, ny in candidates:
            visited[nx][ny] = True
            if backtrack(nx, ny, step + 1):
                return True
            visited[nx][ny] = False
        return False

    visited[sr][sc] = True
    success = backtrack(sr, sc, 1)
    print("success" if success else "fail")

if __name__ == "__main__":
    knight_tour()

```

代码运行截图 (至少包含有"Accepted")

状态: **Accepted**

源代码

```
def knight_tour():
    n = int(input())
    sr, sc = map(int, input().split())
    moves = [(-2, 1), (-1, 2), (1, 2), (2, 1),
              (2, -1), (1, -2), (-1, -2), (-2, -1)]
    visited = [[False] * n for _ in range(n)]
    total_steps = n * n

    def get_degree(x, y):
        count = 0
        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < n and 0 <= ny < n and not visited[nx][ny]:
                count += 1
        return count

    def backtrack(x, y, step):
        if step == total_steps:
            return True
        candidates = []
        for dx, dy in moves:
            nx, ny = x + dx, y + dy
            if 0 <= nx < n and 0 <= ny < n and not visited[nx][ny]:
                degree = get_degree(nx, ny)
                candidates.append((degree, nx, ny))
        candidates.sort()
```

## 2. 学习总结和收获

---

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025spring每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。